R documentation

of all in 'man'

June 13, 2016

R topics documented:

contingencyTable 1
kappa.ct 2
kappa.set 2
rho 3
rho.ct 4
rho.minTSLength 5
rho.set 6
rhoR 7
set 7

Index **8**

`contingencyTable` *contingencyTable sample*

Description

A sample contingencyTable ct = as.data.frame(matrix(c(3,2,1,34), nrow = 2, ncol = 2)) testSet-BaserateInflation = .1

Usage

```
contingencyTable
```

Format

An object of class `data.frame` with 2 rows and 2 columns.

Details

A contingency Table is a 2x2 matrix representing the possible combinations of raters in a set. The contingency Table is represented as a data frame

kappa.ct *Calculate kappa (contingency Table)*

Description

This function calculates kappa given a contingency table and can evaluate if it is greater than a threshold

Usage

```
## S3 method for class 'ct'
kappa(ct, kappaThreshold = NULL)
```

Arguments

ct	A contingencyTable
kappaThreshold	Optional Threshold to compare against, as a double

Value

If kappaThreshold is NULL: kappa as a double If kappathreshold is not NULL: a list containing both the kappa, as a double, and a boolen indicating if it is above the provided threshold.

Examples

```
# Given contingency Table and no threshold
kappa.ct(contingencyTable, isSet = FALSE, kappaThreshold = NULL)

# Given contingency with threshold
kappa.ct(contingencyTable, isSet = FALSE, kappaThreshold = 0.65)
```

kappa.set *Calculate kappa (Set)*

Description

This function calculates kappa given a set and can evaluate if it is greater than a threshold

Usage

```
## S3 method for class 'set'
kappa(set, kappaThreshold = NULL)
```

Arguments

set	A set
kappaThreshold	Optional Threshold to compare against, as a double

Value

If kappaThreshold is NULL: kappa as a double If kappathreshold is not NULL: a list containing both the kappa, as a double, and a boolen indicating if it is above the provided threshold.

Examples

```
# Given set and no threshold
kappa.set(set, isSet = TRUE, kappaThreshold = NULL)

# Given set and threshold
kappa.set(set, isSet = TRUE, kappaThreshold = 0.65)
```

---

rho *Calculate rho*

Description

This function will calculate rho given the parameters of the initial set, which it returns as a double.

Usage

```
rho(observedKappa, codeSetBaserate, testSetLength,
  testSetBaserateInflation = 0, codeSetLength = 10000, runs = 800,
  kappaThreshold = 0.65, nullHypMin = 0.4, precisionMin = 0.6,
  precisionMax = 1)
```

Arguments

observedKappa	The observed kappa value that will be tested with rho, as a double
codeSetBaserate	The frequency of one's for the first rater in the code set, if NULL, assume the baserate of the test set is the same as the code set
testSetLength	The length of the test set for the observedKappa
testSetBaserateInflation	The percent, as a decimal, of the test set that is gauranteed to be a positive for the first rater
codeSetLength	The length of the code set used to calculate rho
runs	The number of code sets to use in the null hypothesis distribution for rho
kappaThreshold	Threshold to compare against, as a double
nullHypMin	The minimum kappa in the null hypothesis distribution for rho
precisionMin	The minimum percision to be used for generation of sets in the null hypothesis distribution for rho
precisionMax	The maximum percision to be used for generation of sets in the null hypothesis distribution for rho

Value

The calcuated rho, as a double

Examples

```
# Given only required parameters
rho(observedKappa = 0.88, codeSetBaserate = 0.2, testSetLength = 80)

# Given required parameters plus testSetBaserateInflation and runs
rho(
 observedKappa = 0.88, codeSetBaserate = 0.2, testSetLength = 80,
 testSetBaserateInflation = 0.2, runs = 500
)
```

`rho.ct` *Rho (contingency Table)*

Description

This function calculates rho and kappa for a given contingency Table, and returns a list containing both values.

Usage

```
rho.ct(table, codeSetBaserate = NULL, testSetBaserateInflation = 0,
  codeSetLength = 10000, runs = 800, kappaThreshold = 0.65,
  nullHypMin = 0.4, precisionMin = 0.6, precisionMax = 1)
```

Arguments

`table`	The `contingencyTable` used to calculate rho
`codeSetBaserate`	The frequency of one's for the first rater in the code set, if NULL, assume the baserate of the test set is the same as the code set
`testSetBaserateInflation`	The percent, as a decimal, of the test set that is gauranteed to be a positive for the first rater
`codeSetLength`	The length of the code set used to calculate rho
`runs`	The number of code sets to use in the null hypothesis distribution for rho
`kappaThreshold`	Threshold to compare against, as a double
`nullHypMin`	The minimum kappa in the null hypothesis distribution for rho
`precisionMin`	The minimum percision to be used for generation of code sets in the null hypothesis distribution for rho
`precisionMax`	The maximum percision to be used for generation of code sets in the null hypothesis distribution for rho

Value

A list containing rho and kappa

Examples

```
# Given testSetBaserateInflation
rho.ct(table, testSetBaserateInflation = 0.2)

# Given codeSetBaserate and testSetBaserateInflation
rho.ct(table, codeSetBaseate = 0.1, testSetBaserateInflation = 0.2)
```

`rho.minTSLength` *Rho Min testSetLength*

Description

This function calculates the minimum testSetLength where it is possible to get a rho less than alpha, which it returns as an integer.

Usage

```
rho.minTSLength(baserate, alpha = 0.05, inc = 10, printInc = FALSE, ...)
```

Arguments

`baserate`	A double indicating the frequency a code occurs for the first rater in the code set
`alpha`	A double indicating the threshold of significance for rho (similar to an alpha level for a p value), defaulted to 0.05
`inc`	An integer indicating by how much the testSetLength should increase each iteration
`printInc`	A boolean indicating whether to print out each increment value with it's corresponding significance for rho
`...`	Any additional parameters passed into `rho`

Value

An integer giving the minimum length of testSet, as a denomination of inc, that would give a value where rho becomes mathematically possible

Examples

```
#Add testSetBaserate as an additional parameter
rho.minTSLength(0.2, testSetBaserate = 0.2)

#Add testSetBaserate as well as changing inc and selecting printInc
rho.minTSLength(0.2, inc = 5, printInc = TRUE, testSetBaserate = 0.2)
```

`rho.set` *Rho (set)*

Description

This function calculates rho and kappa for a given set, and returns a list containing both values.

Usage

```
rho.set(set, codeSetBaserate = NULL, testSetBaserateInflation = 0,
  codeSetLength = 10000, runs = 800, kappaThreshold = 0.65,
  nullHypMin = 0.4, precisionMin = 0.6, precisionMax = 1)
```

Arguments

`set`	The `set` used to calculate rho
`codeSetBaserate`	The frequency of one's for the first rater in the code set, if NULL, assume the baserate of the test set is the same as the code set
`testSetBaserateInflation`	The percent, as a decimal, of the test set that is gauranteed to be a positive for the first rater
`codeSetLength`	The length of the code set used to calculate rho
`runs`	The number of code sets to use in the null hypothesis distribution for rho
`kappaThreshold`	Threshold to compare against, as a double
`nullHypMin`	The minimum kappa in the null hypothesis distribution for rho
`precisionMin`	The minimum percision to be used for generation of sets in the null hypothesis distribution for rho
`precisionMax`	The maximum percision to be used for generation of sets in the null hypothesis distribution for rho

Value

A list containing rho and kappa

Examples

```
# Given testSetBaserateInflation
rho.set(set, testSetBaserateInflation = 0.2)

# Given codeSetBaserate and testSetBaserateInflation
rho.set(set, codeSetBaseate = 0.1, testSetBaserateInflation = 0.2)
```

rhoR *rhoR: A package for computating rho.*

Description

rho is used to test the generalization of inter rater reliability (IRR) statistics.

Details

Calculating rho starts by generating a large number of simulated, fully-coded data sets: a sizable collection of hypothetical populations, all of which have a kappa value below a given threshold - which indicates unacceptable agreement.

Then kappa is calculated on a sample from each of those sets in the collection to see if it is equal to or higher than the kappa in then real sample.

If less than five percent of the distribution of samples from the simulated data sets is greater than actual observed kappa, the null hypothesis is rejected and one can conclude that two raters would have acceptable agreement had they coded all of the rest of the data.

rho

Use rho

kappa

Use kappa

rho.minTSLength

Use rho.minTSLength

---

set *set sample*

Description

A sample set gold = c(1,1,1,1,rep(0,36)) silver = c(1,1,1,0,1,1,rep(0,34)) set = cbind(gold,silver) testSetBaserateInflation = .1

Usage

```
set
```

Format

An object of class `data.frame` with 10000 rows and 3 columns.

Details

A set can be any length of vectors that are bound using cbind the length of each of these vectors becomes the set length

Index

∗Topic **datasets**
contingencyTable, 1
set, 7
∗Topic **kappa.ct**
kappa.ct, 2
∗Topic **kappa.set**
kappa.set, 2
∗Topic **rho.ct**
rho.ct, 4
∗Topic **rho.minTSLength**
rho.minTSLength, 5
∗Topic **rho.set**
rho.set, 6
∗Topic **rho**
rho, 3

cbind, *7*
contingencyTable, 1, *2*, *4*

kappa, *7*
kappa.ct, 2
kappa.set, 2

rho, 3, *5*, *7*
rho.ct, 4
rho.minTSLength, 5, *7*
rho.set, 6
rhoR, 7
rhoR-package *(*rhoR*)*, 7

set, *2*, *6*, 7